

SPECIFY (2026)

Rome Vidal, Robert Rood, Natalie Kite

Abstract

This project implements a full-stack academic scheduling platform designed to optimize student-driven course scheduling through interest aggregation, availability modeling, and automated time-slot optimization. The system allows professors to create courses, students to express interest and submit availability, and the platform to compute optimal meeting times using a scoring-based scheduling engine.

The architecture integrates authentication, real-time dashboard views, CSV reporting, and deployment pipelines. Built with a modular MVP-driven approach, the system evolves through nine incremental milestones including authentication, scheduling logic, UI/UX development, deployment, and testing. The final system is deployed on Vercel with PostgreSQL via Supabase and email services via Resend, enabling a production-ready academic coordination tool.

Executive Summary

Problem Framing

Scheduling inefficiencies in universities have long been a bottleneck for both administrative departments and students. Traditional methods for scheduling special topics courses, such as surveys, spreadsheets, or ad-hoc manual coordination, lead to suboptimal time slot assignments, missed opportunities for course offerings, and, most importantly, an inefficient allocation of time resources. In the absence of an automated decision-support system, departments struggle to balance student demand with faculty availability, and student preferences are often left unconsidered.

The inefficiency is compounded by fragmented data: students' availability is often submitted in disparate formats, ranging from vague time preferences to more structured but inconsistent responses. The traditional, anecdotal nature of availability input makes it difficult to generate meaningful, optimized schedules. Hence, aggregating and analyzing this availability data in a systematic and scalable manner is essential for improving course scheduling.

SpeciFY addresses these issues by automating the collection, aggregation, and analysis of student availability data. By converting this data into a structured model and optimizing it with a scheduling engine, the platform offers universities a more data-driven and accurate solution for organizing special topics courses. This results in more efficient use of faculty time, improved student satisfaction, and a reduction in manual overhead.

High-Level System Table

Component	Purpose
Authentication	Role-based login and session management for Students and Professors
Course System	Creation, modification, and lifecycle management of courses
Interest System	Captures student interest in courses and tracks availability
Scheduling Engine	Converts aggregated availability data into ranked time slots
CSV Export Layer	Reporting layer for exporting scheduling data in CSV format
UI Layer	Dashboards for displaying available courses, ranking data, and export options

System Data Flow

The data flow through the system follows a clear lifecycle:

- Student Interaction:** The process begins when a student logs into the platform, selects a course of interest, and submits their availability (dates, times, etc.).
- Data Aggregation:** This availability data is aggregated and transformed into a SlotAggregate object, which is a structured representation of availability for a given time slot.

3. **Scheduling Engine:** The SlotAggregate is passed to the scheduling engine, which computes availability percentages and ranks time slots based on student interest.
4. **Ranking and Classification:** The system classifies the ranked slots into categories such as "Optimal", "Good", "Risky", and "Disqualified" based on configurable thresholds.
5. **Export/Display:** The final ranked schedule can be viewed by professors on the platform's dashboard, and CSV reports can be exported for further analysis or decision-making.

Deployment Summary

SpeciFY is deployed on the cloud using a serverless model. Vercel is utilized for frontend deployment, enabling efficient scaling and fast content delivery. The PostgreSQL database, managed by Supabase, serves as the primary data store for all course and student information. GitHub Actions facilitates CI/CD workflows, automating deployment processes and ensuring consistent code quality and integration.

The system also integrates with Resend, a service for email notifications, to alert users about course-related events such as course creation, schedule updates, or availability submission deadlines.

Table of Contents

Cover Page	0
Executive Summary	1
Problem Framing	1
High-Level System Table	1
System Data Flow	1
Deployment Summary	2
Table of Contents	3
Abbreviations and Glossary	6
Introduction	8
Motivation	8
Related Conceptual Background	8
Team Background and Prior Work	8
Development Contributions	8
Ownership Statement	9
Development Approach and Major Design Decisions	9
Requirements	10
Requirements Overview	10
Functional Requirements	10
Non-Functional Requirements	11
Derived Requirements	11
Design & Implementation	12
System Architecture	12
Key Design Decisions	12
Module Interaction Overview	13
Mapping Requirements to System Components	13
MVP Breakdown	13
Scheduling Engine Details	14
Development Environment	14
Target Environment	15
Constraint and Design Implications	15

Testing	17
Testing Levels.....	17
Requirement Coverage Matrix.....	18
Testing Summary	18
Deployment	20
System Delivery Model	20
Deployment Infrastructure	20
Deployment Process Overview	20
Phased Rollout Strategy.....	20
Acceptance Testing Summary	21
Summation and Conclusions.....	22
Technical Achievements	22
Challenges.....	22
Lessons Learned.....	23
Future Work.....	23
Acknowledgements	24
References	25
Appendices	26
Appendix A: Requirements	26
A.1 Functional Requirements (FR).....	26
A.2 Non-Functional Requirements (NFR)	26
A.3 Derived Requirements	27
A.4 Requirement Traceability	27
Appendix B: Design	28
B.1 System Architecture.....	28
B.2 Module & Requirements Breakdown.....	29
B.3 Data Models.....	30
B.4 Key Functions / Methods	43
B.5 Authentication	44
B.6 Course Management & Lifecycle State Machine	45
B.7 Interest Tracking, Scheduling Insights & Export Flow	47
B.8 Scheduling Engine	49
B.9 CSV Export System	51

B.10 UI Component Hierarchy	53
B.11 GitHub CI/CD Pipeline	54
B.12 Deployment Architecture.....	56
Appendix C: Tests and Results	58
C.1 Unit Testing.....	58
C.2 Integration Testing.....	60
C.3 Edge Case Testing	61
C.4 Summary of Results	62
Appendix D: Manuals.....	64
D.1 User Manual	64
D.2 Source Code Repository.....	64

Abbreviations and Glossary

A

- **Availability Engine:** The scheduling system component responsible for evaluating and selecting optimal time slots based on aggregated student availability.
- **Availability Matrix:** A structured dataset capturing the availability of students for potential course time slots, used to guide the scheduling process.

C

- **CI/CD:** Continuous Integration / Continuous Deployment pipeline, a system for automating code integration and deployment processes.
- **Course Lifecycle:** The entire lifecycle of a course, from creation and scheduling to completion and closure.
- **CSV:** Comma-Separated Values file format, used for exporting structured data, such as student interest or availability, to CSV files.

D

- **Dashboard:** The role-based UI entry point for users, displaying relevant data and functionalities for students and professors.

G

- **GitHub Actions:** An automation tool for CI/CD workflows, enabling continuous integration, testing, and deployment triggered by repository changes.

H

- **Hydration:** The process by which a React application is initialized and loaded on the client-side from static HTML content previously generated by the server.

I

- **Interest Threshold:** The minimum level of student interest required for a course time slot to be considered viable and eligible for scheduling.
- **Interest Threshold (for Availability):** The minimum student interest needed for a time slot to pass the viability check and move forward to the scheduling process.

M

- **MVP:** Minimum Viable Product milestone, representing the core functionality of the system that can be released to users for early feedback.

N

- **NextAuth:** Authentication system integrated with **Next.js** to manage user sessions securely and support role-based access.

P

- **Prisma ORM:** Object Relational Mapping (ORM) library used to interact with the PostgreSQL database, simplifying data queries and management.

R

- **Resend:** A cloud-based transactional email service used to send application-generated emails via a backend API.
- **Role:** A classification for users in the system, typically denoted as either **STUDENT** or **PROFESSOR**, with specific permissions tied to each role.
- **Role-Based Access Control (RBAC):** A security model that restricts access to system resources based on the user's assigned role (e.g., Student, Professor, Admin).

S

- **Scheduling Engine:** The core component of the system that processes **SlotAggregate** objects, evaluates available time slots, and generates a ranked list of viable course schedules.
- **Server Actions:** A Next.js feature that enables direct invocation of server-side logic from the frontend without separate API routes.
- **SlotAggregate:** A core data structure representing a time slot with aggregated availability data, including the number of students available and unavailable, and the overall percentage of interest for the slot.
- **Supabase:** A hosted PostgreSQL backend platform used to manage data persistence, providing an easy-to-use interface for database operations.

T

- **Threshold:** The minimum level of interest or availability required for a time slot or course to be considered for scheduling or offered to students.
- **Vercel:** A cloud-based deployment platform used for hosting frontend applications and serverless functions, ensuring fast deployment and scalability.

Introduction

Motivation

Scheduling courses in academic settings, especially Special Topics courses, is inherently challenging. These courses often have unpredictable demand, with student interest varying across multiple potential time slots. Additionally, faculty availability often does not align perfectly with student preferences. The result is a complex problem where manual coordination is not only time-consuming but frequently leads to inefficient outcomes. Administrators and professors are left with no clear framework to balance the conflicting needs of students and faculty.

Moreover, traditional scheduling methods that rely on anecdotal student feedback or incomplete survey data fail to accurately represent the broader student body's availability. As a result, many universities fail to offer courses that align with actual student demand, leading to missed opportunities for both students and faculty.

The value of **interest-based course planning** lies in its ability to optimize the scheduling process by using data (specifically, the availability and preferences of students) to guide decision-making. By automating the aggregation and analysis of this data, SpeciFY can provide a more efficient, transparent, and scalable solution to the academic scheduling problem.

Related Conceptual Background

Course scheduling problems are closely related to **demand-based scheduling systems** where the goal is to optimize resources (in this case, course time slots) based on fluctuating demand (student interest). These systems rely on algorithms that balance multiple constraints. SpeciFY leverages an aggregation-based decision system, where individual inputs (student availability) are merged into a collective representation (SlotAggregate) and optimized to produce the best possible scheduling results.

Team Background and Prior Work

The project team consists of Rome (Team Lead), Natalie, and Robert Rood, with academic backgrounds in computer science and cybersecurity.

Rome is a Computer Science student at Roger Williams University who previously earned an Associate's degree in Computer Science from the Community College of Rhode Island. His prior work includes the development of multiple full-stack web applications, a spaceflight planning tool integrating scientific and financial computations, and a content management platform designed to handle large datasets through RESTful APIs and relational databases. He has also contributed to an interactive cybersecurity education application and developed a Windows-based system utility for file management automation.

Natalie is pursuing a dual degree in Computer Science and Cybersecurity, with an academic focus on data analysis and digital forensics. Robert Rood is a Cybersecurity major with interests in threat analysis and system security.

These academic backgrounds provided general familiarity with software engineering principles, data handling, and security considerations during the planning and design phases of SpeciFY.

Development Contributions

Although the project was structured as a three-person team, implementation work was primarily carried out by the lead developer. Other team members contributed during early-stage discussions, conceptual planning, and review of design ideas. The final system implementation, integration, and deployment were completed by the primary developer.

Ownership Statement

The SpecIFY Scheduling System is an original software application developed as part of this project. All system architecture, implementation, and associated artifacts were created during the development process documented in this report.

The project was developed independently and does not incorporate proprietary institutional software, code, or infrastructure.

At the time of submission, the system and its source code are maintained under the control of the primary developer, Joel G. Vidal (Rome). The system is currently deployed at specify.tools and may be transferred to a future legal entity (Zero Downtime LLC) upon formal registration.

Development Approach and Major Design Decisions

The development of SpecIFY followed an Agile-inspired methodology, emphasizing iterative development, continuous feedback, and incremental feature delivery. Given the evolving nature of the problem (balancing student availability with faculty constraints), development prioritized flexibility over rigid planning. This allowed requirements to be refined throughout implementation as new insights emerged from testing and system behavior.

Rather than adhering strictly to a linear model such as Waterfall, development proceeded in short cycles organized around feature-based milestones. Core system components, such as authentication, availability submission, and the scheduling engine, were developed and validated incrementally. This approach reduced risk by enabling early identification of design issues, particularly in the aggregation and ranking logic central to the system.

A Minimum Viable Product (MVP) strategy guided early development decisions. Initial efforts focused on implementing essential functionality (user roles, course interaction, and availability aggregation) before expanding into features such as CSV export and ranking thresholds. This ensured a functional system early in development and supported iterative refinement.

Additionally, key architectural decisions were made early in the project. A full-stack JavaScript framework (Next.js) was selected to unify frontend and backend development, reducing complexity and improving maintainability. A serverless deployment model was chosen to support scalability and minimize infrastructure management overhead. These decisions directly influenced how requirements were structured and implemented, as described in the following section.

Requirements

Requirements Overview

The primary goal of SpeciFY is to assist academic administrators and faculty in making data-driven scheduling decisions for courses with uncertain demand. Specifically, the system aims to collect student availability and interest data, aggregate that information, and generate optimized course scheduling recommendations that balance student demand with faculty constraints.

The requirements for the system were derived from this core objective, focusing on enabling efficient data collection, processing, and interpretation. Early in development, the team identified the need to support two primary user groups (students and professors) each with distinct interactions with the system. This led to the definition of role-based functionality and guided the separation of system features.

As development progressed, several requirements evolved. For example, initial designs focused solely on collecting availability data, but this expanded to include ranking and classification of time slots to provide more actionable insights. Similarly, export functionality was introduced later to support external analysis and administrative reporting needs.

Defining requirements presented several challenges. One major difficulty was balancing simplicity in user interaction with the complexity of the underlying scheduling logic. Additionally, determining meaningful thresholds for ranking time slots required iterative refinement to ensure that results were both accurate and useful. These challenges led to adjustments in both functional and non-functional requirements throughout the development process.

Functional Requirements

- **FR1:** The system shall allow users to authenticate via institutional credentials.
- **FR2:** The system shall support two roles, STUDENT and PROFESSOR, each with different permissions.
- **FR3:** Professors shall be able to create, manage, and update course information, including defining course schedules and availability.
- **FR4:** Students shall be able to browse courses and express interest in courses they wish to participate in.
- **FR5:** Students shall be able to submit their availability for the courses they are interested in.
- **FR6:** The system shall aggregate student availability data for processing by the scheduling engine.
- **FR7:** The system shall compute and rank viable course time slots based on availability data and defined criteria.
- **FR8:** The system shall provide export functionality in CSV format, including summary, detailed, and recommendation formats.

Non-Functional Requirements

- **Performance:** The system must efficiently handle scheduling data for N students and M courses without performance degradation.
- **Security:** The system shall enforce RBAC for user authentication and authorization to protect sensitive data.
- **Reliability:** The system must provide deterministic outputs when processing the scheduling logic to ensure consistent results.
- **Usability:** The platform shall have a responsive, intuitive UI that is easy to navigate for both students and professors.

Derived Requirements

- **Slot Aggregation:** The SlotAggregate model must compute the availability percentage for each time slot, and the system must rank these slots based on student participation metrics.
- **CSV Export:** The CSV export functionality must support multiple formats: a summary report, a detailed report of individual student preferences, and a recommended schedule.

Design & Implementation

System Architecture

SpeciFY is designed as a full-stack, serverless web application following a layered architecture model. This approach separates concerns across presentation, application logic, and data management, improving maintainability and scalability. A high-level architecture diagram (see Appendix B) illustrates the interaction between these layers and external services:

- **Frontend (UI Layer):** Built using Next.js with React and Tailwind CSS. The frontend handles user interaction, displaying data and facilitating navigation between different components based on user roles.
- **Backend (Application Layer):** Next.js API routes handle the scheduling logic, including aggregating availability data into SlotAggregate objects and running the scheduling engine.
- **Data Layer:** PostgreSQL, managed by Supabase, stores all user, course, and availability data. Prisma ORM is used to interact with the database.
- **External Services:** The system integrates with Resend for email notifications and Vercel for cloud-based deployment.

Key Design Decisions

Several key architectural and design decisions guided the development of SpeciFY:

- **Full-Stack Framework (Next.js):**
A unified framework was selected to handle both frontend and backend functionality. This reduced context switching, simplified deployment, and ensured tight integration between UI and server-side logic.
- **Serverless Architecture:**
A serverless model was chosen to minimize infrastructure management and enable automatic scaling. This is particularly beneficial for handling variable workloads, such as fluctuating student usage during course selection periods.
- **Layered Architecture Pattern:**
The system separates concerns into distinct layers (UI, application logic, data), improving modularity and making the system easier to maintain and extend.
- **Aggregation-Based Scheduling Model:**
Instead of attempting complex optimization algorithms, the system uses an aggregation and ranking approach. This simplifies computation while still producing meaningful scheduling recommendations.
- **Relational Database (PostgreSQL):**
A structured data model was required to manage relationships between users, courses, and availability, making a relational database the most appropriate choice.

Module Interaction Overview

The system operates through a structured flow of data between components:

- 1. Users interact with the frontend interface to submit availability or manage courses.
- 2. The frontend makes requests through backend Server Actions.
- 3. The backend processes requests, invoking business logic such as scheduling and aggregation.
- 4. The scheduling engine retrieves and processes data from the database.
- 5. Results are returned to the frontend for visualization or export.

This interaction model ensures a clear separation between user interaction, business logic, and data management.

Mapping Requirements to System Components

System components were designed to directly support the functional requirements defined earlier.

Authentication and role-based access control (FR1–FR2) are handled through the authentication and authorization layer.

Course management and student interaction features (FR3–FR5) are implemented within the application layer and exposed through the user interface.

The scheduling engine (FR6) is responsible for aggregating and ranking availability data, while the export module (FR7) enables generation of CSV reports.

Non-functional requirements such as performance and reliability are addressed through the system’s architecture, including the use of serverless infrastructure and deterministic processing logic.

MVP Breakdown

The system uses a modular, MVP-driven architecture that separates concerns across distinct layers to support maintainability and scalability. Each layer can be developed and modified independently, reducing coupling and enabling the system to evolve without major structural changes.

Layer	MVP Responsibility
Presentation Layer	User Interface, Role-based dashboards
Application Layer	Scheduling Engine, Business Logic
Data Layer	Server Actions, PostgreSQL (Supabase), Prisma ORM
Infrastructure Layer	Deployment (Vercel), CI/CD (GitHub Actions)

This separation keeps scheduling logic independent from the UI and isolates data and deployment concerns from application logic, resulting in a clear, testable, and scalable system structure.

Scheduling Engine Details

The scheduling engine converts raw availability data into structured, ranked outputs. The process involves:

1. **SlotAggregate Transformation:** Availability data is grouped by time slot and aggregated into SlotAggregate objects, which hold metrics like total students, available students, unavailable students, and availability percentages.
2. **Ranking Logic:** Each SlotAggregate is assigned a ranking based on its availability metrics, and slots are classified as "Optimal," "Good," "Risky," or "Disqualified."
3. **Threshold Classification:** The system uses configurable thresholds to categorize the viability of each time slot.

Development Environment

The SpecIFY system was developed using a modern full-stack JavaScript ecosystem designed to support rapid development, maintainability, and scalability. The primary development environment consisted of a unified framework (Next.js) combined with TypeScript for type safety and reliability.

Development Workflow:

The team followed a feature-branch workflow with pull request-based development. All changes were reviewed before merging to ensure code quality and maintain system stability.

Key tools and technologies used:

- **Next.js** for full-stack application development (frontend and backend)
- **TypeScript** for static typing and improved code quality
- **Tailwind CSS** for responsive and consistent UI design
- **HeroUI** for prebuilt, accessible UI components that accelerate interface development
- **Lucide** for lightweight, customizable iconography used throughout the interface
- **PostgreSQL** (via **Supabase**) for relational data storage
- **Prisma ORM** for database interaction
- **Vitest** for unit and integration testing
- **GitHub Actions** for continuous integration and deployment
- **Vercel CLI** for deployment and environment management

These tools were selected to streamline development workflows, reduce configuration overhead, and support a cohesive full-stack architecture.

Target Environment

The SpecIFY system is deployed as a cloud-based web application designed for accessibility and scalability.

- **Browser-Based Execution Environment:**

The application is designed to run on modern web browsers, such as Chrome, Firefox, and Safari, which provide full support for the features required by SpecIFY, including JavaScript, CSS, and HTML5 standards.

- **Cloud Deployment on Vercel**

The entire frontend of the application is deployed on **Vercel**, a platform that provides cloud-based hosting, automatic scaling, and serverless function support. Vercel enables rapid deployment and ensures that the application remains highly available and performs well under varying traffic loads.

- **Database on Supabase**

The **PostgreSQL database** is managed by **Supabase**, a fully managed platform for building and deploying databases. Supabase ensures that the database is securely hosted, easily accessible, and scalable. All user, course, and availability data is stored in Supabase, and Prisma ORM is used to interact with the database.

- **Serverless API Model**

The backend logic for the application is built on a **serverless architecture**, where API routes in Next.js handle requests for scheduling logic, course management, and data aggregation. The serverless model reduces overhead and scales automatically with demand, ensuring cost efficiency and flexibility.

This environment enables the system to handle variable workloads efficiently while minimizing infrastructure management overhead.

Constraint and Design Implications

Several constraints influenced the design and implementation of the system:

- **Serverless Execution Constraints:**

The use of serverless functions required the scheduling logic to be efficient and stateless, influencing the design of the aggregation and ranking process.

- **Browser-Based Access:**

Since the system operates entirely in the browser, the UI needed to be responsive and performant across devices.

- **Scalability Requirements:**

The system was designed to support large numbers of users (e.g., thousands of students), requiring efficient database queries and aggregation logic.

- **Time and Scope Constraints:**

As a semester-based project, development prioritized a functional MVP, influencing the selection of technologies that reduce setup complexity and accelerate development.

Testing

This section outlines the testing strategy used to validate the SpeciFY Scheduling System. The testing approach is designed to ensure correctness, consistency, and completeness across all functional and non-functional requirements. Validation is performed across multiple levels, ranging from isolated unit-level verification to full system and edge-case testing.

The overall objective of the testing strategy is to confirm that all system requirements are satisfied and that the scheduling engine, authentication system, and data export processes behave deterministically and reliably under expected and unexpected conditions.

Testing Levels

To ensure system reliability, a structured multi-level testing strategy is employed:

1. Unit Testing

Unit testing is used to validate individual functions and isolated logic components within the system. These tests focus on ensuring that core computational behavior is correct at the smallest functional level.

A significant portion of the testing effort focuses on **server actions**, which represent core backend operations in the application. These actions are tested in isolation using Vitest with mocked database and framework dependencies to ensure predictable and repeatable results.

Key areas include:

- Server actions for user management (e.g., user creation, lookup, and validation logic)
- Server actions for course management (e.g., creation, retrieval, updates, and deletion)
- Prisma-based database interaction logic, tested through mocked query interfaces
- Authentication-related logic, including role assignment and user verification
- Scheduling utilities responsible for ranking and classifying time slots
- Data transformation logic converting raw availability into SlotAggregate structures
- CSV formatting functions ensuring consistent export output

Mocking strategies are used extensively to isolate logic from external dependencies such as the database layer and Next.js runtime features (e.g., redirects and cache revalidation).

2. Integration Testing

Integration testing verifies the interaction between multiple system components to ensure correct data flow across the architecture.

Key areas include:

- Frontend dashboard integration with backend APIs for dynamic scheduling updates

- Authentication system integration with role-based access control (RBAC)
- Scheduling engine integration with database queries and UI rendering
- Interaction between server actions and the database layer (via mocked Prisma clients during test execution)
- Verification that server actions correctly trigger expected side effects, such as redirects or cache revalidation

These tests ensure that independently tested units function correctly when composed into higher-level workflows.

3. Edge Case Testing

Edge case testing ensures system stability under abnormal or extreme conditions.

Scenarios include:

- Empty availability datasets (no student interest submitted)
- Tie-breaking scenarios in scheduling rankings
- Invalid authentication inputs or unrecognized email domains
- Failure cases in database operations (e.g., rejected Prisma promises)
- Conditional logic paths such as test-user detection and fallback creation flows

Requirement Coverage Matrix

Requirement	Tested	Status
Scheduling correctness	Yes	Pass
Data aggregation and ranking	Yes	Pass
Server action user management	Yes	Yes
Server action course management	Yes	Yes
CSV export formatting	Yes	Pass
Role-based access control	Yes	Pass
User authentication	Yes	Pass

This matrix demonstrates full traceability between system requirements and testing coverage, confirming that all major functional requirements have been validated.

Testing Summary

The system has been validated across all major functional and non-functional requirements using a structured testing strategy focused heavily on server-side logic validation.

Results confirm that:

- Core server actions behave deterministically under mocked conditions
- User and course management operations execute correctly across all tested scenarios
- Core scheduling logic produces deterministic and consistent outputs
- Authentication and role-based access control behave as expected
- Database interactions are correctly invoked and handled through Prisma-based abstractions
- Data aggregation and export functionality operate correctly across all tested scenarios
- Edge cases, including missing data and unauthorized access, are handled safely without system failure

Overall, the testing process confirms that the SpecifY system meets its intended design requirements at the backend logic level and demonstrates strong reliability in its server-side operations.

Deployment

System Delivery Model

SpeciFY is delivered as a cloud-based web application and does not require local installation. End users access the system through a web browser, where the application is hosted on Vercel. The system is deployed using a serverless architecture, allowing both frontend and backend functionality to be executed within a unified platform.

The deployment process is automated through a CI/CD pipeline integrated with GitHub and Vercel. Code changes are continuously tested, built, and deployed, ensuring that the latest stable version of the application is always available.

Deployment Infrastructure

The application is hosted on Vercel, which provides automated deployment triggered by changes merged into the main branch. Environment-specific variables, including API keys and database credentials, are securely managed through Vercel's configuration system. Built-in monitoring tools support performance tracking and issue detection after deployment.

The system uses a managed PostgreSQL database hosted on Supabase. Database schema changes are applied through Prisma migrations to ensure consistency and integrity across development, staging, and production environments.

Deployment Process Overview

The system follows an automated deployment workflow:

- Code changes are developed in feature branches and submitted via pull requests
- Automated tests are executed to validate functionality
- Approved changes are merged into the development branch and deployed to a staging environment
- After validation, changes are merged into the main branch, triggering production deployment on Vercel

This process ensures consistent, repeatable deployments while minimizing manual intervention.

Phased Rollout Strategy

The deployment of SpecifY followed a phased approach:

1. **Development Phase:**
Core features, including authentication, course management, and availability submission, were implemented and tested locally.
2. **Staging Phase:**
Features were deployed to a staging environment via the development branch, allowing for integration testing and validation of system behavior.

3. **Production Phase:**

After successful validation, the system was deployed to production via the main branch, making it accessible to end users.

This phased rollout reduced deployment risk and allowed issues to be identified and resolved prior to production release.

Acceptance Testing Summary

Acceptance testing was conducted to verify that the system satisfies the functional and non-functional requirements defined in earlier sections.

- **Functional Validation:**

Core features such as user authentication, role-based access control, course management, availability submission, and scheduling generation were tested to ensure correct behavior.

- **Scheduling Engine Verification:**

The aggregation and ranking logic was validated using sample datasets to ensure that outputs were consistent, accurate, and aligned with expected results.

- **Export Functionality:**

CSV export features were tested to confirm that generated files matched system data across all supported formats.

- **Non-Functional Validation:**

The system was tested for usability, responsiveness, and reliability, ensuring consistent performance across different browsers and deterministic scheduling outputs.

All major requirements were successfully met, and the system was deemed ready for deployment.

Summation and Conclusions

Technical Achievements

The SpeciFY Scheduling System successfully addresses the complexity of academic course scheduling by providing an automated, data-driven approach to aligning student availability with course time slots.

A key achievement of the system is the design of a structured scheduling model built around TimeSlot and TimeBlock representations. These abstractions enable consistent handling of scheduling data across multiple days and time ranges, allowing identical time intervals to be distinguished by context while still supporting aggregation. This design provides the foundation for accurate availability tracking, aggregation, and downstream scheduling computations.

Another major achievement is the implementation of an interactive schedule input grid featuring click-and-drag functionality. This interface allows users to efficiently select and modify availability across multiple time slots in a single interaction. The grid-based approach improves usability while ensuring that input data remains structured and consistent with the underlying TimeSlot model.

The scheduling engine builds on these inputs by aggregating student availability data into structured SlotAggregate objects and applying a ranking mechanism to evaluate and prioritize viable time slots. The engine classifies results into four categories—Optimal, Good, Risky, and Disqualified—based on configurable thresholds, enabling flexible and deterministic decision-making.

The system also includes a CSV export pipeline that allows scheduling data to be exported in multiple formats, including summary, detailed, and recommendation views. This supports administrative workflows by making scheduling outcomes easily accessible and interpretable.

Finally, the platform implements a role-based architecture that enforces secure and structured access to features. Students and professors interact with tailored interfaces, ensuring that each user type engages only with relevant functionality such as course interaction, availability submission, and scheduling analysis.

Challenges

Several challenges were encountered during development, particularly in handling the structure, consistency, and interpretation of scheduling input and output data.

One major challenge involved representing time slots consistently across multiple days. Identical time ranges (e.g., 10:00–11:00) needed to be treated as distinct entities depending on the day of the week, while still allowing meaningful aggregation and comparison. Designing a model that preserved this distinction without introducing redundancy or ambiguity required careful structuring of the TimeSlot and SlotAggregate relationships.

Another challenge was implementing reliable availability versioning. As students updated their availability prior to course finalization, the system needed to track changes without overwriting prior states. This required a versioned snapshot approach, ensuring that scheduling computations always operated on the correct and most recent dataset while preserving historical consistency.

A further difficulty was ensuring accurate and interpretable output through availability aggregation and heatmapping. Translating raw availability data into meaningful visual or ranked outputs required consistent percentage calculations, stable aggregation logic, and clear categorization thresholds. The system needed to ensure that similar inputs always produced identical outputs, reinforcing reliability and trust in the scheduling results.

Lessons Learned

This project highlighted several important software engineering principles that will inform future development work.

A key lesson was the value of modular system design. Separating scheduling logic from UI components significantly improved maintainability and made it easier to extend or modify core functionality without impacting other parts of the system.

Another important takeaway was the importance of clear separation between business logic and presentation layers. This separation improved system scalability and reduced complexity during debugging and feature expansion.

The implementation of a CI/CD pipeline also reinforced the benefits of automated testing and deployment. It improved development efficiency, reduced deployment errors, and ensured consistent integration of new features.

Future Work

While the current system provides a strong foundation for automated scheduling, several enhancements could further improve its functionality, usability, and scalability.

One potential improvement is the introduction of email notifications to keep users informed of important events such as course updates, scheduling results, and submission deadlines. This would improve communication between students and professors and increase overall engagement with the platform.

Another area for expansion is the addition of an analytics dashboard. Professors and administrators could gain insights into trends such as student availability patterns, course demand, and participation rates, enabling more informed decision-making and long-term planning.

Finally, implementing an admin dashboard would extend system control and oversight. Administrators could manage users, monitor system activity, adjust scheduling thresholds, and oversee course lifecycle states, improving maintainability and governance of the platform.

Acknowledgements

This section identifies individuals and tools that supported the development of the Specify Scheduling System. Tool and service usage does not imply contribution to system design or implementation.

Team Members:

- Rome Vidal (Team Lead)
- Robert Rood
- Natalie Kite

These individuals participated in early-stage discussions, conceptual planning, and design feedback during the development of the system.

Open-Source Tools and Frameworks:

- **Next.js:** Full-stack React framework used for frontend and backend development.
- **Prisma ORM:** Used for database access and schema management with PostgreSQL.
- **Tailwind CSS:** Utility-first CSS framework used for responsive and consistent UI styling.
- **Vitest:** Testing framework used for unit and integration testing across the application.

Platforms & Services

- **Vercel:** Cloud platform used for hosting and deploying the application with automated CI/CD integration.
- **Supabase:** Managed PostgreSQL database platform used for persistent data storage.
- **GitHub Actions:** CI/CD automation tool used for testing and deployment workflows.

UI Libraries

- **HeroUI:** UI component library used to accelerate frontend development and maintain design consistency.
- **Lucide:** Icon library used to provide consistent and scalable iconography across the interface.

References

To ensure that the development of SpecIFY was informed by best practices and current technologies, the following resources were referenced throughout the project:

Frameworks & Libraries

- **Next.js Documentation:** <https://nextjs.org/docs>
- **React.js Documentation:** <https://react.dev>
- **Prisma ORM Documentation:** <https://www.prisma.io/docs>
- **Supabase Documentation:** <https://supabase.com/docs>
- **PostgreSQL Documentation:** <https://www.postgresql.org/docs>
- **Vercel Documentation:** <https://vercel.com/docs>
- **Vitest Documentation:** <https://vitest.dev>
- **NextAuth Documentation:** <https://next-auth.js.org>
- **Tailwind CSS Documentation:** <https://tailwindcss.com/docs>
- **GitHub Actions Documentation:** <https://docs.github.com/en/actions>
- **HeroUI Documentation:** <https://heroui.com/>
- **Lucide Icons Documentation:** <https://lucide.dev>

Appendices

The following appendices provide detailed supporting documentation that complements the content in the main report. This includes specifications, design details, testing outcomes, and other relevant material to ensure full clarity and transparency of the **SpeciFY Scheduling System**.

Appendix A: Requirements

This section outlines the detailed requirements of the **SpeciFY Scheduling System**. Each requirement is specified in terms of functionality, performance, security, and usability, along with the associated metrics for determining successful implementation.

A.1 Functional Requirements (FR)

ID	Requirement	Description	Acceptance Criteria
FR1	Authentication System	The system shall allow users to authenticate using institutional credentials.	Users can successfully log in and log out; invalid credentials are rejected; session persists securely.
FR2	Role-Based Access Control	The system shall support distinct roles (student, professor).	Users cannot access professor features; professors can access course management tools.
FR3	Course Creation	Professors shall create/manage courses.	Courses can be created, edited, and deleted; changes persist in database.
FR4	Interest & Availability	Students submit interest and availability.	Availability submissions are stored and retrievable per course.
FR5	Availability Aggregation	System aggregates availability data.	Aggregated results correctly reflect submitted data with no loss or duplication.
FR6	Scheduling Optimization	System computes ranked schedules.	Output includes ranked time slots with consistent ordering based on input data.
FR7	CSV Export	System exports scheduling data.	CSV files download successfully and match system data in all formats.

A.2 Non-Functional Requirements (NFR)

ID	Requirement	Description	Acceptance Criteria
NFR1	Performance	System handles large datasets	Processes scheduling for 50 students within ≤ 2 seconds

NFR2	Security	RBAC enforcement	Unauthorized access attempts are denied
NFR3	Usability	Intuitive UI	Users complete key tasks (submit availability, create course) without guidance in usability testing
NFR4	Reliability	Deterministic outputs	Same input dataset produces identical outputs across runs

A.3 Derived Requirements

Derived From	Requirement	Description
FR5, FR6	Availability Aggregation	Must compute percentages and ranking scores
FR7	CSV Export Formats	Must support multiple structured output formats

A.4 Requirement Traceability

Requirement ID	System Component
FR1	Authentication Module (NextAuth, Server Actions, Resend)
FR2	Authentication Module (NextAuth, Server Actions, UI)
FR3	Course Creation & Browsing Module
FR4	Interest Tracking Module
FR5	Scheduling Engine (Aggregation Layer)
FR6	Scheduling Engine (Ranking Logic)
FR7	CSV Export Service
NFR1	Backend Optimization / Database Queries
NFR2	Authentication + Authorization Layer
NFR3	Frontend UI (React + Tailwind)
NFR4	Scheduling Engine Design

Appendix B: Design

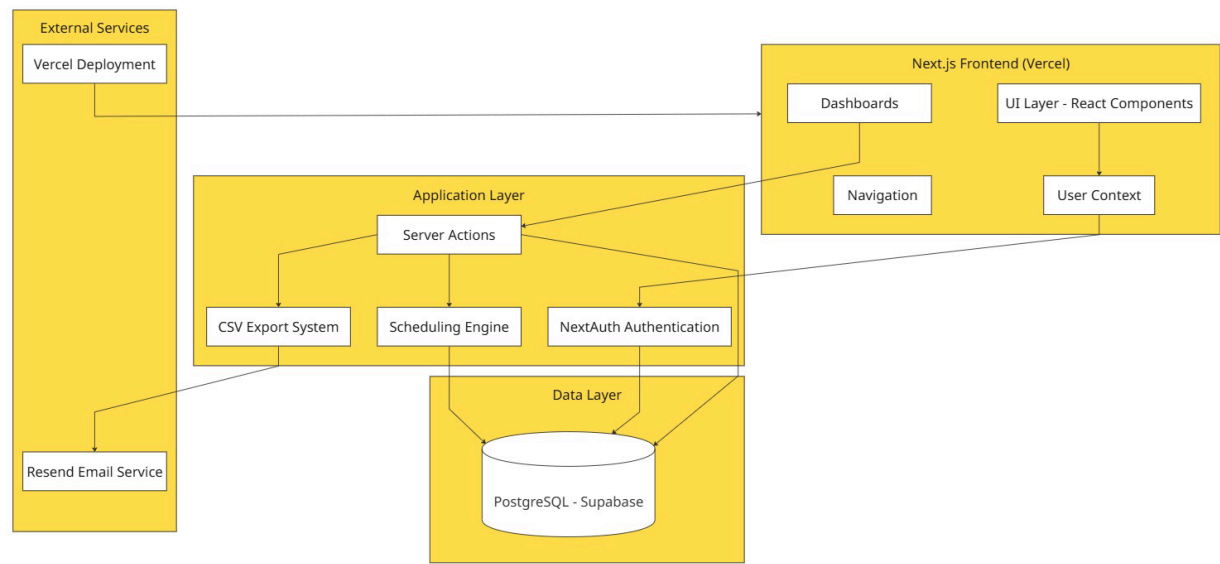
This section presents the detailed design of the **SpeciFY Scheduling System**, including data models, architecture diagrams, and the modules required to meet the project’s requirements. Each design choice is linked to specific functional and non-functional requirements.

B.1 System Architecture

The **SpeciFY Scheduling System** follows a modular, layered architecture. The components of the system are divided into four primary layers:

Layer	Responsibilities
Presentation Layer	Handles UI components using React (Next.js).
Application Layer	Contains the core business logic, including the Scheduling Engine .
Data Layer	Persists data in PostgreSQL via Supabase.
Infrastructure Layer	Manages deployment through Vercel , automated testing with GitHub Actions , and email services with Resend .

Diagram: High-Level System Architecture



This diagram illustrates the interaction between different components of the system, including the frontend, backend, database, and external services. The system follows a clear flow from the frontend UI to the backend services, and finally to the database and external email service

B.2 Module & Requirements Breakdown

The system is divided into several key modules, each responsible for a specific part of the system's functionality. Below are the modules and their responsibilities:

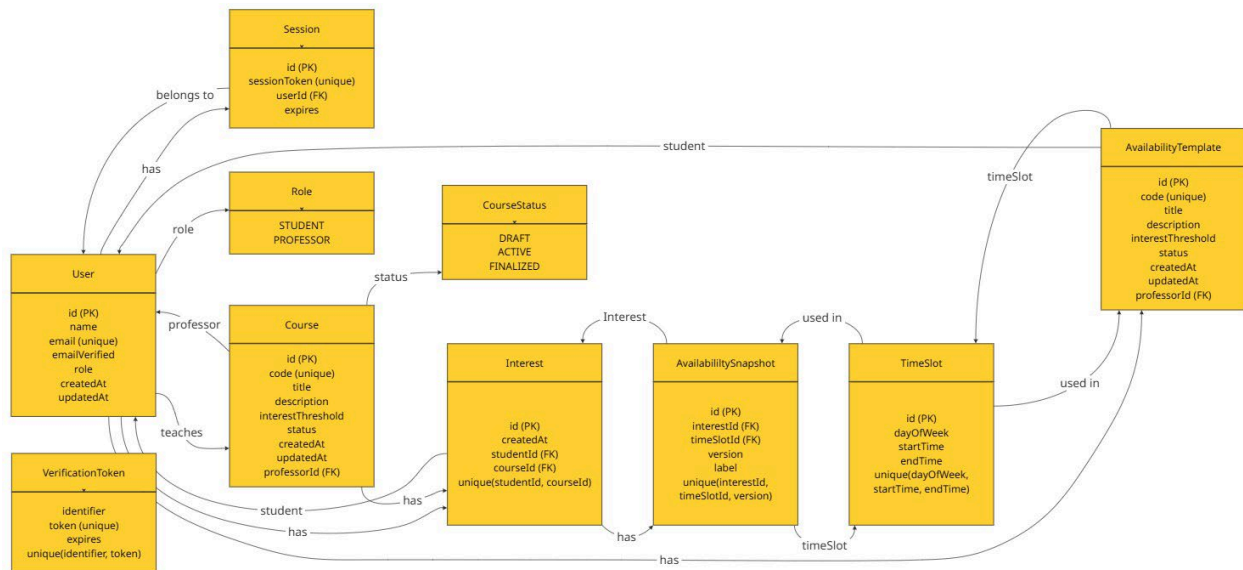
Module	Responsibilities
Authentication	Manages role-based login, session handling, and user state (via NextAuth)
Course Management	Handles creation, editing, and managing courses by professors
Interest Tracking	Records student interest and availability data for courses
Scheduling Engine	Aggregates student availability into SlotAggregate objects and ranks potential time slots
CSV Export	Provides functionalities for exporting scheduling data in various CSV formats
UI/UX	Renders role-based dashboards and manages interactions

Each system requirement maps directly to a corresponding module as follows:

Requirement	Design Component
FR1	Authentication Module
FR2	RBAC / User Context
FR3	Course Management Module
FR4	Interest Tracking Module
FR5	Scheduling Engine (Aggregation Layer)
FR6	Scheduling Engine (Ranking Logic)
FR7	CSV Export System
NFR1	Database + Backend Optimization
NFR2	Auth + RBAC Layer
NFR3	UI Layer
NFR4	Deterministic Scheduling Engine

B.3 Data Models

B.3.1 Data Relationships



User & Authentication

- A User can have many **Session** records (FR1, NFR2)
- Each **Session** belongs to one **User** (FR1, NFR2)

Course Management

- A User (**Professor**) can teach many Course records (FR3, FR2)
- A Course is associated with one **Professor** (FR3, FR2)
- A Course can have many **Interest** records (FR3, FR4)

Student Interest

- A User (**Student**) can have many Interest records (FR4, FR2)
- An Interest belongs to one **Student** (FR4)
- An Interest belongs to one **Course** (FR4, FR3)

Availability System

- A User (Student) can have many **AvailabilityTemplate** records (FR4, NFR1)
- Each **AvailabilityTemplate** belongs to one **Student** (FR4)
- Each **AvailabilityTemplate** references one **TimeSlot** (FR5)
- A **TimeSlot** can be referenced by many **AvailabilityTemplate** records (FR5, NFR1)
- An Interest can have many **AvailabilitySnapshot** records (FR5)
- Each **AvailabilitySnapshot** belongs to one **Interest** (FR5)
- Each **AvailabilitySnapshot** references one **TimeSlot** (FR5)
- A **TimeSlot** can be referenced by many **AvailabilitySnapshot** records (FR5, NFR1)

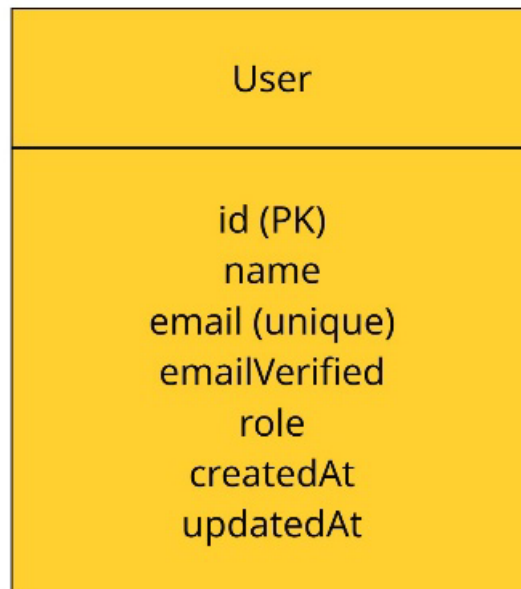
Derived / Aggregated Data

- A Course has many **AvailabilitySnapshot** records (indirectly through Interest) (FR5)
- Each **AvailabilitySnapshot** contributes to **SlotAggregate** calculations (FR5, FR6)
- A **SlotAggregate** represents aggregated data for one **TimeSlot** across many students (FR5, FR6, NFR4)

B.3.2 Models

The system's data models represent key entities in the application. Below is a detailed overview of the major data models used:

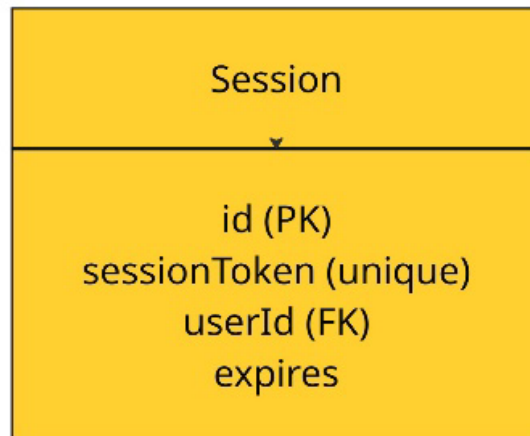
User Model



The **User** model represents all users in the system, including students and professors:

- **UserID:** Unique identifier.
- **Name:** User's full name.
- **Email:** Unique email address.
- **EmailVerified:** Timestamp indicating when the email was verified.
- **Role:** Defines whether the user is a student or professor.
- **CreatedAt:** Timestamp of account creation.
- **UpdatedAt:** Timestamp of last update.
- **Courses:** Courses taught (if the user is a professor).
- **Interests:** Courses the user is interested in (if a student).
- **Sessions:** Active authentication sessions.
- **AvailabilityTemplates:** Default availability preferences for the user.

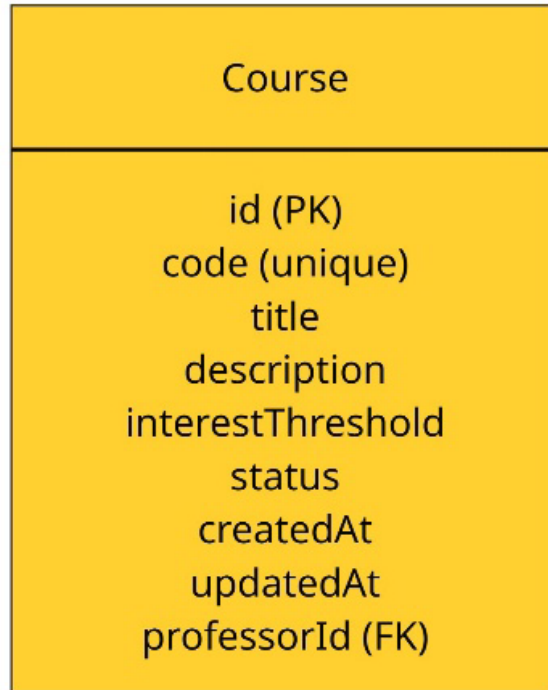
Session Model



The **Session** model manages user authentication sessions:

- **SessionID**: Unique identifier.
- **SessionToken**: Unique session token.
- **UserID**: Reference to the associated user.
- **Expires**: Expiration timestamp of the session.

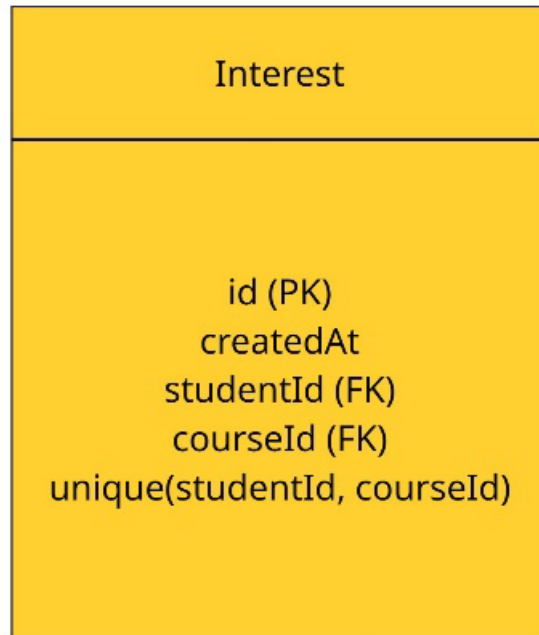
Course Model



The **Course** model represents course-specific information, including:

- **CourseID**: Unique identifier for the course.
- **Code**: Unique course code.
- **Title**: Name of the course.
- **Description**: Detailed course description.
- **InterestThreshold**: Minimum interest required for activation.
- **Status**: Current state (Draft, Active, Finalized).
- **CreatedAt**: Timestamp of creation.
- **UpdatedAt**: Timestamp of last update.
- **ProfessorID**: Reference to the professor teaching the course.
- **Interests**: Students who have expressed interest in the course.

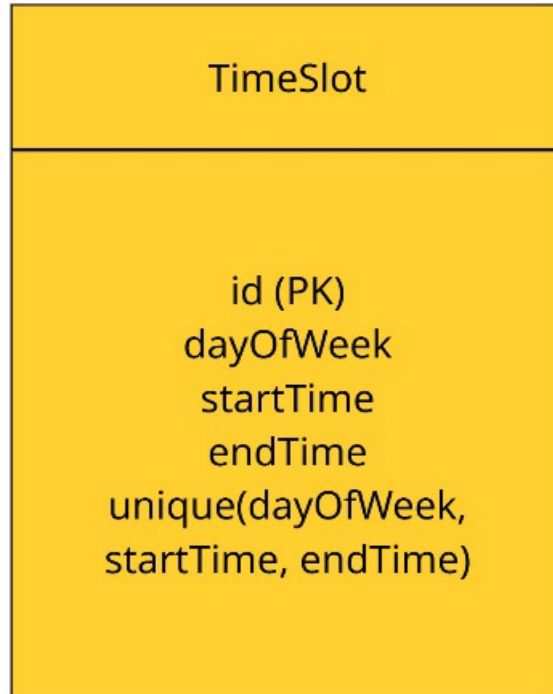
Interest Model



The **Interest** model links students to courses they express interest in:

- **InterestID:** Unique identifier.
- **CreatedAt:** Timestamp of creation.
- **StudentID:** Reference to the student who expressed interest.
- **CourseID:** Reference to the course.
- **AvailabilitySnapshots:** Captures the student's availability for the course over time.

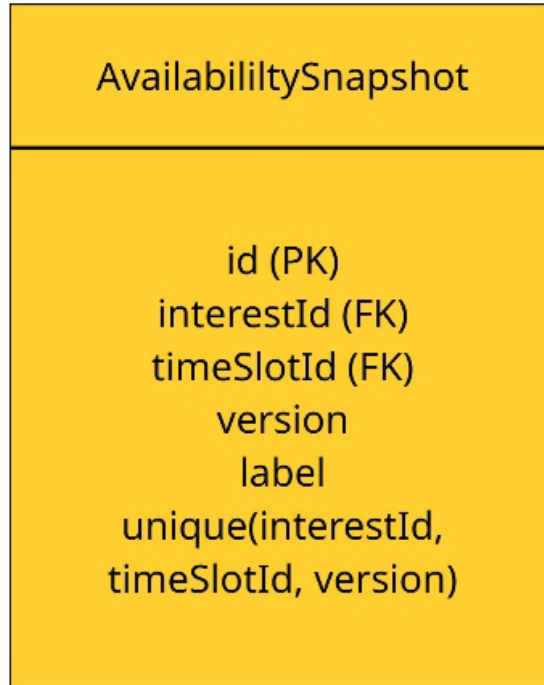
TimeSlot Model



The **TimeSlot** model defines standardized time intervals:

- **TimeSlotID**: Unique identifier.
- **DayOfWeek**: Numeric representation of the day.
- **StartTime**: Start time of the slot.
- **EndTime**: End time of the slot.
- **AvailabilityTemplates**: Templates that use this time slot.
- **AvailabilitySnapshots**: Snapshots that reference this time slot.

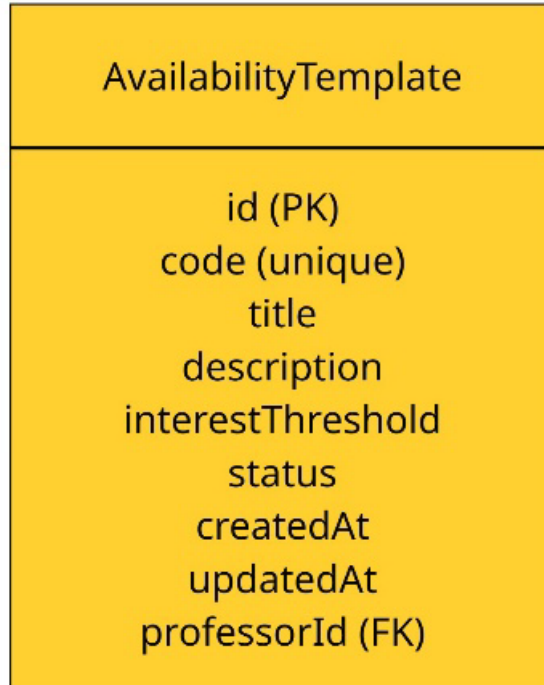
AvailabilitySnapshot Model



The **AvailabilitySnapshot** captures a versioned record of availability data for a specific course time slot:

- **SnapshotID:** Unique identifier.
- **InterestID:** Reference to the associated interest.
- **TimeSlotID:** Reference to the time slot.
- **Version:** Version number to track changes over time.
- **Label:** Optional label for the availability state.

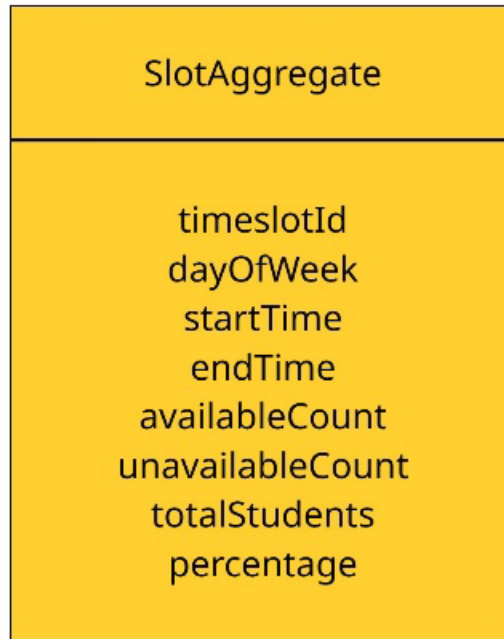
AvailabilityTemplate Model



The **AvailabilityTemplate** model stores a student's default availability preferences:

- **TemplateID:** Unique identifier.
- **StudentID:** Reference to the student.
- **TimeSlotID:** Reference to the time slot.
- **Label:** Optional label for the availability (e.g., "Preferred").

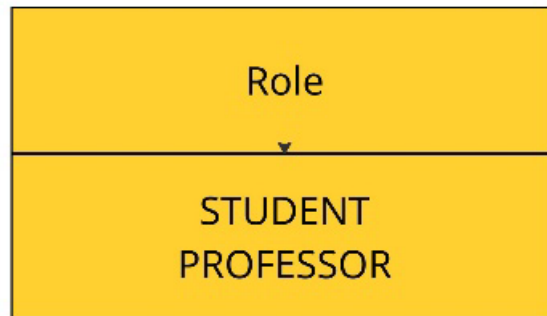
SlotAggregate Model



The **SlotAggregate** model is a derived, in-memory structure computed from AvailabilitySnapshot data that represents a time slot with aggregated student availability. It includes:

- **TimeSlotID:** Reference to the time slot.
- **DayOfWeek:** Day of the week.
- **StartTime:** Start time of the slot.
- **EndTime:** End time of the slot.
- **AvailableStudentIDs:** List of student IDs who are available.
- **UnavailableStudentIDs:** List of student IDs who are unavailable.
- **AvailableCount:** Number of available students.
- **UnavailableCount:** Number of unavailable students.
- **TotalStudents:** Total number of students considered.
- **Percentage:** Percentage of students available.
- **AvailableStudents:** Detailed list of available students (ID, name, email).
- **UnavailableStudents:** Detailed list of unavailable students (ID, name, email).

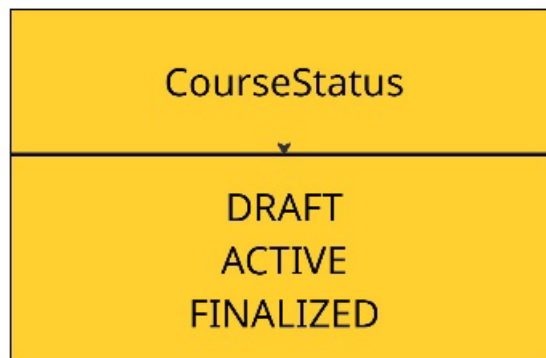
Role (Enum)



The **Role** enum defines the type of user in the system:

- **STUDENT:** A user who can express interest in courses and provide availability.
- **PROFESSOR:** A user who can create and manage courses.

CourseStatus (Enum)

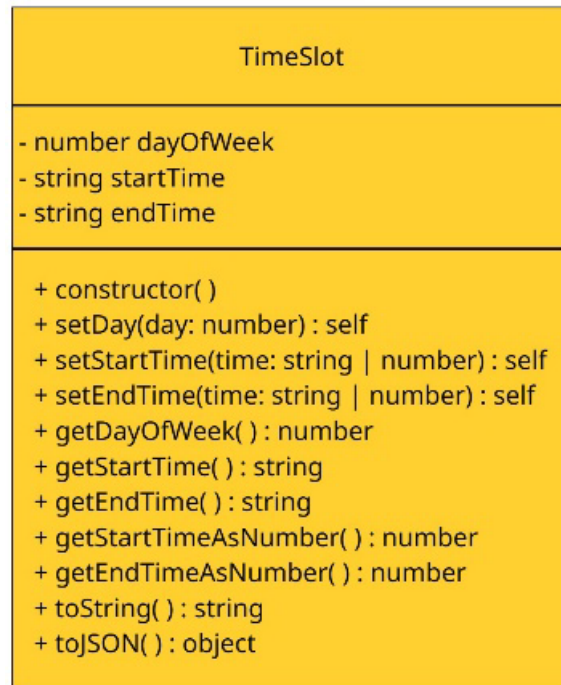


The **CourseStatus** enum represents the lifecycle state of a course:

- **DRAFT:** The course is being created and is not yet visible to students.
- **ACTIVE:** The course is open for student interest and participation.
- **FINALIZED:** The course has been finalized, typically after scheduling decisions are made.

B.3.3 Class Diagrams

TimeSlot Class



The **TimeSlot** class represents a single standardized hour-based interval within a weekly schedule, providing structured day and time formatting for use in scheduling and aggregation logic.

TimeBlock Class

TimeBlock
- string id - Day[] days - number startTime - number endTime - TimeBlockType type - string label - string courseCode - string courseTitle - boolean logger
- log(message: string, property: string, day?: string) - stringToDay(day: string) : Day - dayToNumber(day: string Day) : number + getId() : string + getDays() : Day[] + getStartTime() : number + getAdjustedStartTime() : number + getEndTime() : number + getAdjustedEndTime() : number + getType() : TimeBlockType + getTypeAsString() : string + getLabel() : string + getCourseCode() : string + getCourseTitle() : string + getStartPeriod() : Period + getEndPeriod() : Period + setId(id: string) : TimeBlock + setStartPeriod(period: string) : TimeBlock + setEndPeriod(period: string) : TimeBlock + setDays(days: string[]) : TimeBlock + setStartTime(startTime: number) : TimeBlock + setEndTime(endTime: number) : TimeBlock + setType(type: string) : TimeBlock + setLabel(label: string) : TimeBlock + setCourseCode(courseCode: string) : TimeBlock + setCourseTitle(courseTitle: string) : TimeBlock + toggleDay(day: string) : TimeBlock + hasDay(day: string) : boolean + isValidDay(day: string) : boolean + length() : number + compareTo(timeblock: TimeBlock) : boolean + toString() : string + toJSON() : object + toTimeSlotsArray() : TimeSlot[] + toTimeSlotsObjectArray() : object[] + deepCopy() : TimeBlock

The **TimeBlock** class encapsulates schedule blocks and provides conversion utilities to discrete TimeSlot representations, enabling compatibility with the scheduling engine.

B.4 Key Functions / Methods

Authentication

- `ensureUser(email, name)`: verifies user exists and returns a normalized email address (FR1, NFR2)
- `auth( )`: returns session object (FR1, NFR2)

Course Management

- `createCourse(data)`: creates a new course in the database (FR3)
- `getCourses( )`: returns all courses, including their interests and professor (FR3)
- `getCourseById( )`: returns a specific course, including its interests and professor (FR3)
- `updateCourse(id, date)`: updates a course's data (FR3)
- `updateCourseStatus(status, id)`: updates a course's status (FR3)
- `deleteCourse(id)`: finds and deletes a specified course (FR3)

Interest Tracking

- `addInterest(courseId, timeslots)`: adds a user's interest in a specific course (FR4)
- `removeInterest(courseId)`: removes a user's interest in a specific course (FR4)

Scheduling Engine

- `getCourseAvailabilityData(courseId)`: fetches course data for the scheduling engine (FR5, NFR1)
- `applyPreferences(data, preferences)`: returns filtered slots (FR5, FR6)
- `getTopSlots(data, n)`: returns the top n TimeSlots (FR6, NFR4)
- `getBottomSlots(data, n)`: returns the bottom n TimeSlots (FR6, NFR4)

CSV Export

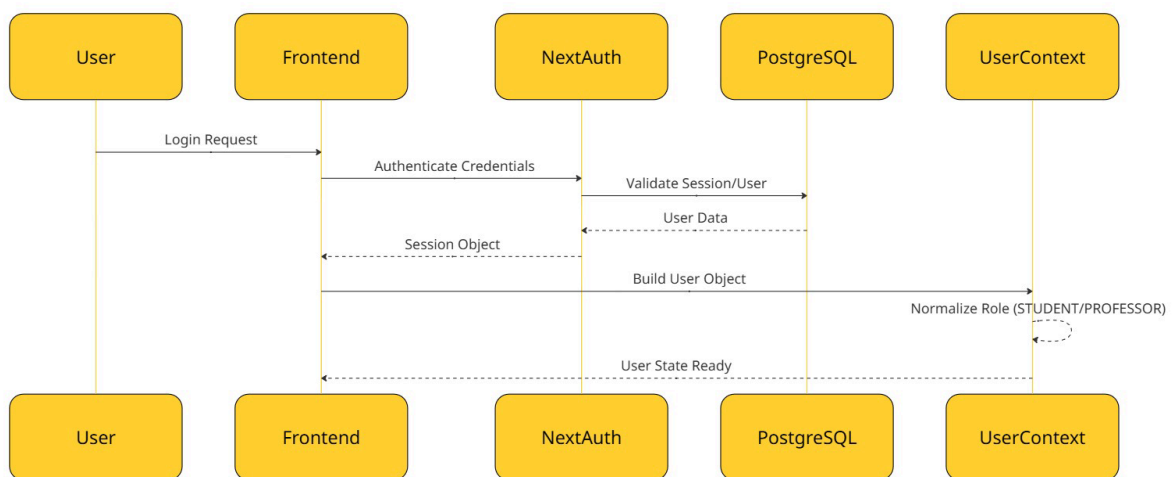
- `downloadRecommendationsFile(filteredSlots, data)` (FR7)
- `downloadSummaryFile(filteredSlots, data)` (FR7)
- `downloadDetailedFile(filteredSlots, data)` (FR7)

B.5 Authentication

The authentication module is responsible for managing user identity, session handling, and role-based access control within the SpecIFY system. It ensures that only authenticated users can access protected resources and that application behavior is dynamically adapted based on the user's role (Student or Professor).

The system uses NextAuth to handle authentication and session management. Upon successful login, user credentials are validated against the PostgreSQL database, and a session object is generated. This session is then used to construct a normalized user context within the frontend, which determines feature access and UI rendering based on role permissions.

Diagram: User Context & Authentication Flow



This diagram illustrates the authentication pipeline from initial user login through credential validation, session creation, and frontend context initialization. It highlights how NextAuth interacts with the PostgreSQL database to validate users and how the resulting session is transformed into a structured user context within the frontend.

The authentication process follows a structured sequence to ensure secure and consistent access control across the system:

- Users initiate a login request through the frontend interface
- NextAuth handles credential validation and session creation
- The system queries the PostgreSQL database to retrieve user identity and role information
- A session object is returned to the frontend upon successful authentication
- The frontend constructs a normalized **UserContext** based on session data
- Role-based access control is enforced by adjusting available routes and UI components dynamically

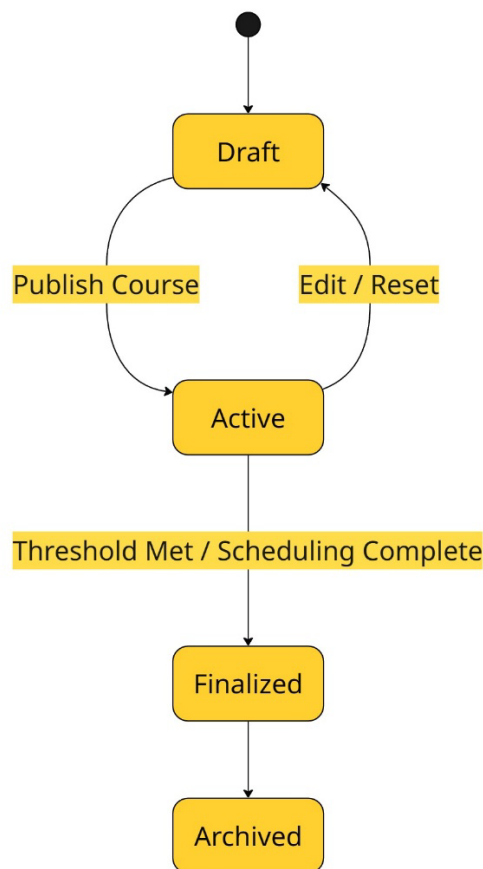
This design ensures that authentication logic remains centralized while allowing the frontend to enforce role-specific behavior in a scalable and maintainable way.

B.6 Course Management & Lifecycle State Machine

The course management module is responsible for handling the creation, modification, and lifecycle progression of academic courses within the SpecIFY system. It ensures that courses move through a controlled and predictable set of states, enforcing consistency in how courses are created, published, scheduled, and finalized.

At the core of this module is a finite state machine that governs all course transitions. This state machine enforces system integrity by restricting invalid transitions and ensuring that downstream processes, such as scheduling and interest aggregation, only operate on courses in valid states.

Diagram: Course Lifecycle State Machine



This diagram represents the valid states of a course and the allowed transitions between them, ensuring structured progression from creation to completion.

B.6.1 Lifecycle State Behavior

The course lifecycle is implemented as a constrained state machine to ensure data consistency and enforce business rules across the system. Each state represents a distinct phase in the course lifecycle:

- **Draft**
 - Initial state when a course is created
 - Course is not visible to students
 - Allows unrestricted editing by professors
- **Active**
 - Course is published and visible to students
 - Students can express interest and submit availability
 - Scheduling engine begins processing relevant data
- **Finalized**
 - Course scheduling has been completed
 - Time slot recommendations are locked
 - No further modifications to scheduling inputs are permitted
- **Archived**
 - Final terminal state
 - Course is stored for historical reference and reporting

B.6.2 State Transition Rules

The system enforces strict rules for state transitions:

- Courses must be explicitly published to move from **Draft** → **Active**
- Courses transition to **Finalized** only after scheduling completion or threshold satisfaction
- Only finalized courses can be archived
- Certain transitions (e.g., Active → Draft) are allowed only for controlled editing or reset operations

B.6.3 Design Rationale

Using a state machine model ensures that course behavior remains predictable and consistent across the system. It prevents invalid operations such as scheduling inactive courses or modifying finalized results, while also providing a clear structure for integrating scheduling logic and student interaction workflows.

This design choice improves maintainability by isolating lifecycle rules into a single controlled mechanism, rather than distributing state logic across multiple system components.

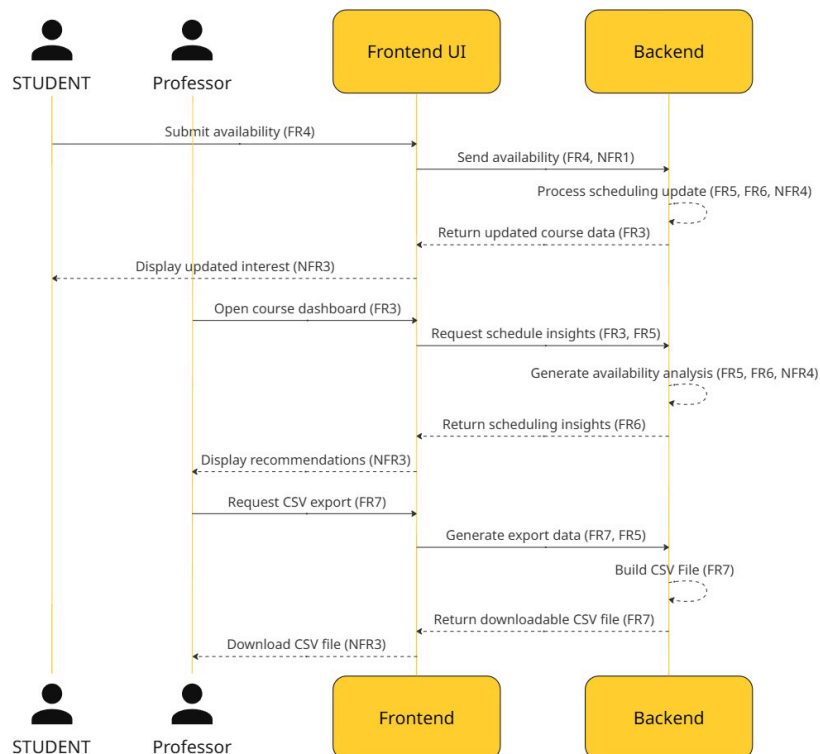
B.7 Interest Tracking, Scheduling Insights & Export Flow

This module coordinates student engagement, scheduling computation, and result distribution into a unified workflow. It acts as the primary interaction layer between user-driven input (interest and availability submission) and backend scheduling logic, ensuring that all generated outputs remain consistent across the system.

Student availability submissions and course interest data are processed by the backend scheduling engine, which generates ranked time slot recommendations. Professors access aggregated scheduling insights derived from the same dataset, ensuring both roles operate on a shared source of truth.

In addition, the module provides export functionality that allows both students and professors to generate structured CSV reports for external use, including analysis, reporting, and administrative decision-making.

Diagram: Scheduling Insights & Export Flow



This diagram illustrates the interaction between students, professors, the frontend interface, and the backend API across three key workflows: availability submission, scheduling insight generation, and CSV export. All processes are routed through a centralized backend computation layer to ensure consistency between displayed scheduling results and exported data.

B.7.1 Workflow Breakdown

The module is organized into three interconnected workflows:

1. Student Engagement Workflow

- Students submit course availability through the frontend interface
- The backend processes availability updates and integrates them into the scheduling engine
- Updated scheduling results are returned and reflected in the UI in real time

2. Professor Insight Workflow

- Professors access course dashboards to view aggregated scheduling data
- The backend generates scheduling insights based on student availability
- Ranked recommendations are returned for administrative decision-making

3. Export Workflow

- Both students and professors can request CSV exports of scheduling data
- The backend generates structured CSV files from scheduling outputs
- Files are returned as downloadable artifacts for external use

B.7.2 Design Rationale

This module centralizes all interactions between user input and scheduling output to ensure a consistent data flow across the system. By routing both student and professor actions through the same backend scheduling pipeline, the system avoids data duplication and ensures that exported results always reflect the latest computed scheduling state.

This design also improves maintainability by consolidating scheduling-related interactions into a single coordinated workflow layer rather than distributing logic across multiple modules.

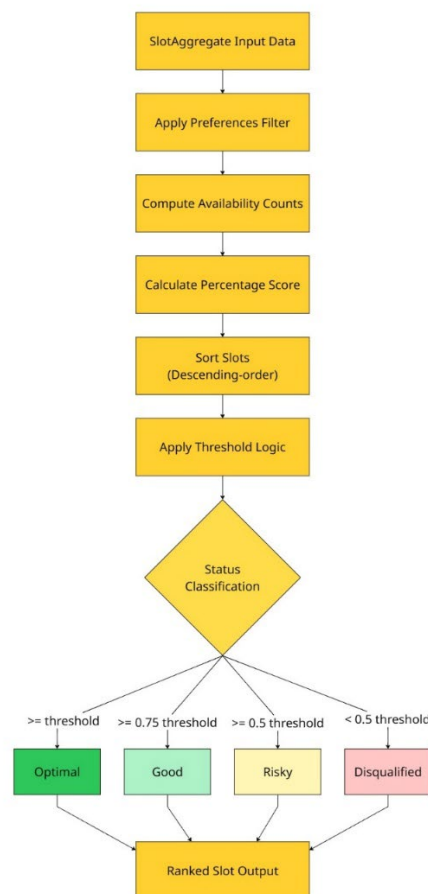
B.8 Scheduling Engine

The scheduling engine is the core computational module of the SpeciFY system. It is responsible for transforming raw student availability data into structured scheduling recommendations through a deterministic ranking pipeline.

The engine operates on pre-processed availability inputs and converts them into SlotAggregate objects, which represent aggregated availability metrics for each time slot. These aggregates form the basis for all scheduling decisions within the system.

Once constructed, each SlotAggregate is evaluated using a scoring and ranking mechanism that determines the relative suitability of time slots. The system applies a threshold-based classification model to categorize each slot into one of four levels: Optimal, Good, Risky, or Disqualified. This ensures that scheduling output remains consistent, interpretable, and repeatable across identical input datasets.

Diagram: Scheduling Engine Pipeline



This diagram illustrates the full scheduling computation pipeline, beginning with aggregated availability data and progressing through filtering, scoring, ranking, and classification. The final output is a ranked set of time slots that represent optimized scheduling recommendations based on student availability.

B.8.1 Pipeline Stages

The scheduling engine is structured as a deterministic multi-stage pipeline:

1. Input Aggregation

Raw availability data is transformed into SlotAggregate objects, consolidating student availability per time slot.

2. Preference Filtering

User-defined constraints and availability preferences are applied to refine the dataset before scoring.

3. Metric Computation

Each time slot is evaluated based on:

- total student availability
- unavailable counts
- overall participation percentage

4. Scoring & Ranking

Slots are sorted in descending order based on computed availability scores, establishing a preliminary ranking.

5. Threshold Classification

A rule-based system classifies each slot into predefined categories:

- **Optimal:** highest viability for scheduling
- **Good:** strong candidate slots
- **Risky:** partially viable slots
- **Disqualified:** insufficient availability

B.8.2 Design Rationale

The scheduling engine is designed as a deterministic pipeline to ensure that identical input data always produces consistent output. This is critical for maintaining trust in scheduling recommendations across both students and professors.

The use of a threshold-based classification system provides interpretability, allowing users to understand not only the ranking of time slots but also the reasoning behind their categorization. Additionally, separating computation into distinct pipeline stages improves modularity, maintainability, and future extensibility (e.g., integrating optimization or machine learning models).

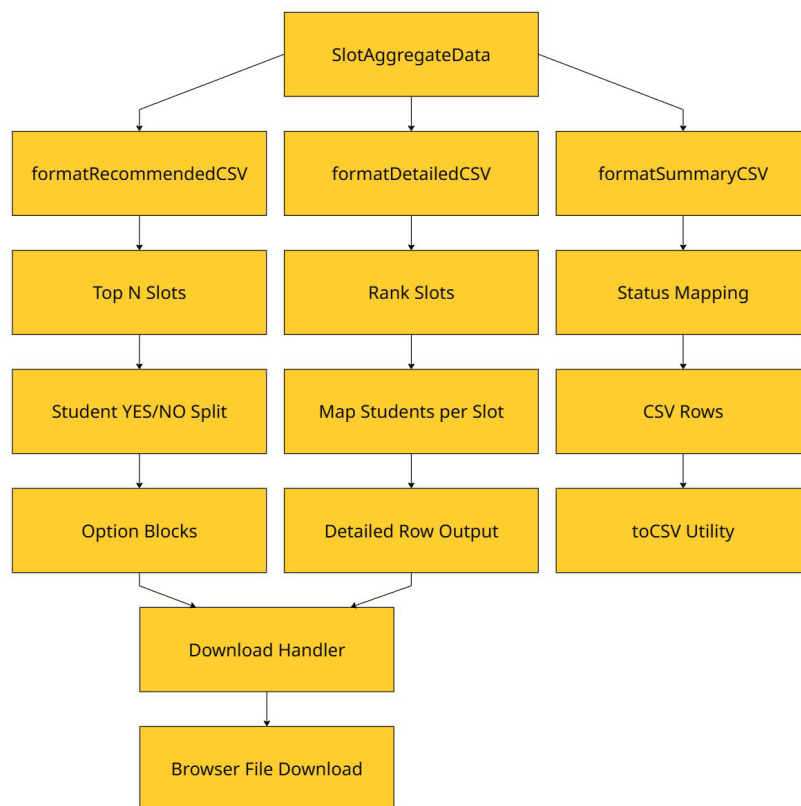
B.9 CSV Export System

The CSV export system is responsible for transforming computed scheduling results into structured, downloadable reports for administrative and analytical use. It serves as the final output layer of the scheduling pipeline, converting SlotAggregate data into multiple standardized CSV formats.

The module ensures consistency across all exports by deriving every format from the same underlying SlotAggregate dataset. This design guarantees that summary, detailed, and recommendation views all reflect the same scheduling state, preventing discrepancies between exported reports and displayed system data.

The export system is implemented as a modular formatting pipeline, where each export type applies a distinct transformation strategy before being passed through a unified CSV generation handler in the backend.

Diagram: CSV Export System Architecture



This diagram illustrates the transformation pipeline from a unified SlotAggregate dataset into three distinct CSV generation strategies. Each strategy applies a different formatting logic before passing the processed data into a shared CSV utility responsible for file generation and download delivery.

B.9.1 Export Strategy Breakdown

The system supports three primary export formats, each designed for a specific use case:

1. Summary Export

- Provides high-level scheduling overview
- Maps slot status classifications into simplified rows
- Used for quick administrative review

2. Recommended Export

- Focuses on top-ranked scheduling options
- Includes filtered slot sets based on ranking thresholds
- Separates student availability into structured preference groups

3. Detailed Export

- Provides full breakdown of scheduling data
- Includes per-slot student mappings and ranking metadata
- Used for in-depth analysis and auditing

B.9.2 Design Rationale

The CSV export system is designed as a unified transformation layer to ensure consistency between computed scheduling results and exported data representations. By deriving all export formats from a single `SlotAggregate` source, the system eliminates duplication and ensures that all reports reflect identical underlying state.

Additionally, separating export logic into distinct formatting strategies improves modularity, allowing new export types to be introduced without modifying core scheduling logic or backend computation processes.

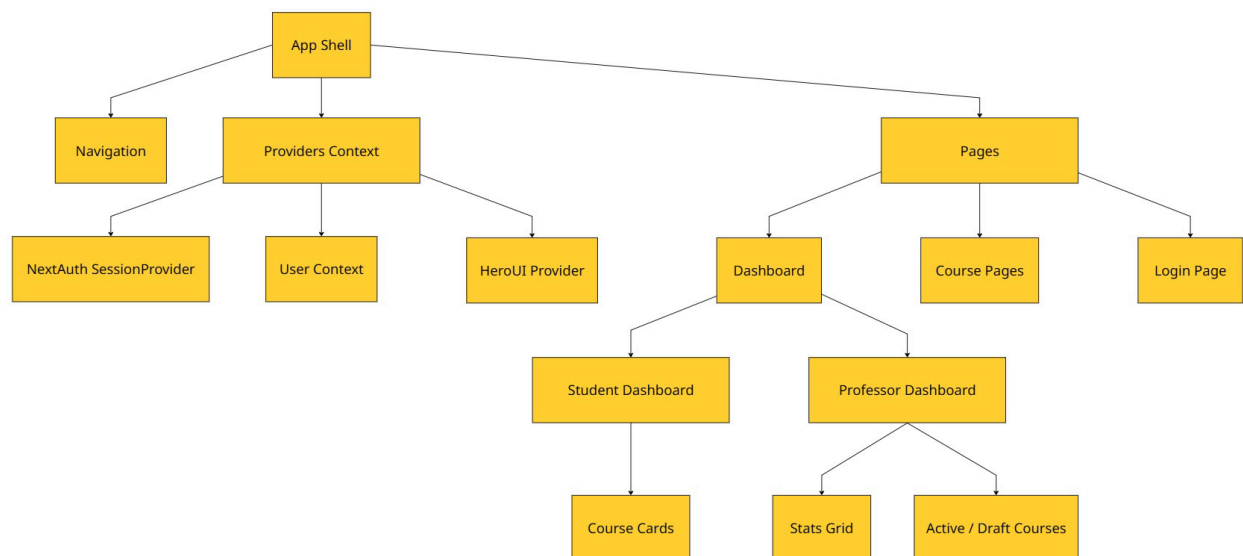
B.10 UI Component Hierarchy

The UI is organized using a component-based architecture, where reusable elements are composed to build higher-level views such as dashboards and course pages.

The system leverages **HeroUI** to provide prebuilt, accessible UI components (e.g., cards, buttons, modals), enabling consistent design patterns across the application. These components are integrated within custom React components to support role-specific views, such as student and professor dashboards.

Additionally, **Lucide** is used for iconography throughout the interface. Icons are embedded within navigation elements, buttons, and status indicators to improve usability and visual clarity without introducing heavy dependencies.

Diagram: UI Component Hierarchy



This diagram details the structure of the frontend UI components and their interrelationships. It shows how different UI elements, such as dashboards, course pages, and login pages, interact with each other within the React (Next.js) application.

The component hierarchy follows a top-down structure:

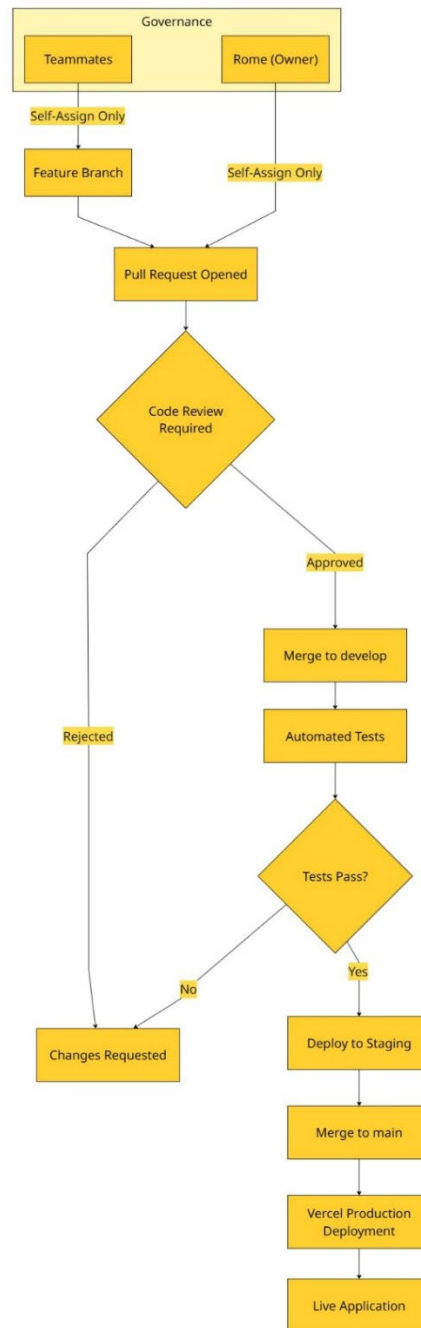
- The **App Shell** manages global layout and navigation
- **Context Providers** (e.g., authentication and user context) supply application-wide state
- **Page Components** define route-level views (Dashboard, Courses, Login)
- **Feature Components** (e.g., Course Cards, Statistics Panels) implement core functionality
- **UI Components** (HeroUI-based) provide reusable building blocks

This layered composition improves maintainability and allows UI elements to be reused consistently across different parts of the system.

B.11 GitHub CI/CD Pipeline

The GitHub CI/CD pipeline automates the build, testing, and deployment processes for the SpecIFY system, ensuring code quality and reducing manual intervention.

Diagram: GitHub CI/CD & Governance Pipeline



This diagram illustrates how feature branches are reviewed, tested, and merged into staging and production branches, with deployment happening automatically on successful tests.

The pipeline is triggered whenever a pull request is opened or updated. Each pull request undergoes automated testing using Vitest to verify functionality and prevent regression. Only code that passes all tests and satisfies review requirements is eligible for merging.

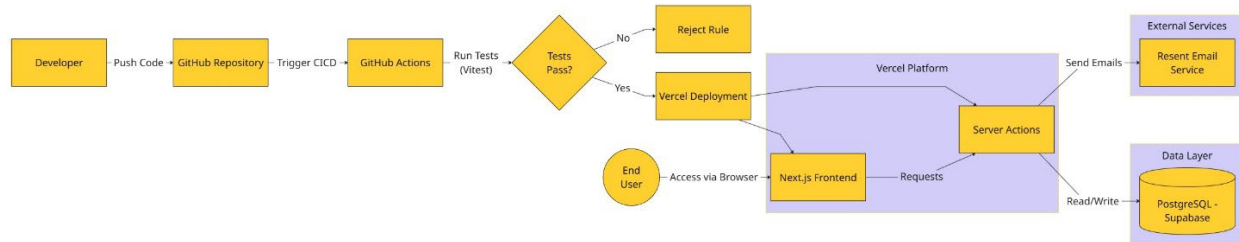
The workflow follows a structured governance model:

- Developers create **feature branches** for isolated development
- Changes are submitted via **pull requests**
- Each pull request requires **peer review approval** before merging
- Approved changes are merged into a development branch and automatically tested
- Successful builds are deployed to staging, followed by production deployment via Vercel

This pipeline ensures that all changes are validated before reaching production, supporting the system's reliability and maintainability requirements.

B.12 Deployment Architecture

Diagram: Deployment Architecture



This diagram illustrates the production deployment structure of the SpecIFY system and the relationships between the application, CI/CD pipeline, database, and external services.

The system is deployed using a serverless architecture on Vercel, which hosts both the Next.js frontend and API routes. The application is automatically built and deployed from the GitHub repository through a CI/CD pipeline powered by GitHub Actions.

B.12.1 Core Deployment Components

- **Vercel:** Hosts the frontend and executes backend API routes as serverless functions
- **GitHub Repository:** Serves as the source of truth for application code and triggers CI/CD workflows
- **GitHub Actions:** Executes automated testing and validation prior to deployment
- **Supabase (PostgreSQL):** Provides managed database services for persistent data storage
- **Prisma ORM:** Handles database schema migrations and ensures consistency across environments
- **Resend:** Handles transactional email delivery (e.g., notifications, exports)

B.12.2 Deployment Flow

1. Code is pushed to the GitHub repository
2. GitHub Actions runs automated tests and validation
3. On successful build, Vercel deploys the updated application
4. The application connects to Supabase for data access and Resend for email services

This architecture enables automatic scaling, high availability, and minimal infrastructure management, aligning with the system's performance and reliability requirements.

B.12.3 Environment Structure

The system is deployed across three environments:

- **Development:** Local feature development and testing
- **Staging:** Pre-production validation via the develop branch
- **Production:** Live environment deployed from the main branch on Vercel

Each environment uses separate configuration variables managed through Vercel's environment system.

Configuration Notes

- Environment variables (e.g., database URL, API keys) are injected via Vercel's deployment settings
- Database schema updates are applied using Prisma migrations
- No manual server configuration is required due to the serverless deployment model

Appendix C: Tests and Results

This appendix provides detailed test cases and results used to validate the SpecIFY Scheduling System. The tests confirm that all functional and non-functional requirements were successfully implemented and behave as expected.

Testing includes unit-level validation of server-side logic, integration-style verification of component interactions through mocked dependencies, and edge case evaluation. Each test case is mapped to expected system behavior and verified through execution results.

C.I Unit Testing

Unit tests validate isolated components of the system to ensure correctness of core logic. Testing is implemented using **Vitest**, with extensive use of mocking to isolate server actions from external dependencies such as the database and framework runtime.

Server Action Tests (User Management)

Test Case	Input	Expected Output	Actual Result	Status
Test user upsert	Test email (prof/student pattern)	User upserted and verified	As expected	Pass
New user creation	Valid email + name	User created in database	As expected	Pass
Missing user with no name	Valid email only	Error thrown	As expected	Pass
Existing user lookup	Existing email	User returned without creation	As expected	Pass
Email normalization	Uppercase email input	Normalized lowercase email	As expected	Pass
Role derivation	Valid email domain	Correct role assigned	As expected	Pass
Database failure handling	Prisma create error	Custom error thrown	As expected	Pass

Server Action Tests (Course Management)

Test Case	Input	Expected Output	Actual Result	Status
Fetch all courses	None	Courses ordered by createdAt	As expected	Pass
Fetch course by ID	Course ID	Matching course returned	As expected	Pass
Create course	Valid course data	Course created + revalidation triggered	As expected	Pass
Update course	Course ID + data	Course updated correctly	As expected	Pass
Delete course	Course ID	Course deleted + revalidation triggered	As expected	Pass
Update course status	Status + ID	Status updated correctly	As expected	Pass
Remove interest (authorized)	User + course ID	Interest removed + redirect triggered	As expected	Pass
Remove interest (unauthorized)	No user	Error thrown	As expected	Pass
Availability data aggregation	Course ID	Structured availability object returned	As expected	Pass
Missing course handling	Invalid course ID	Error thrown	As expected	Pass

Scheduling Engine Tests

Test Case	Input	Expected Output	Actual Result	Status
Slot aggregation calculation	Sample availability dataset	Correct SlotAggregate with accurate counts and percentages	As expected	Pass
Ranking classification	SlotAggregate list	Slots classified as Optimal/Good/Risky/Disqualified	As expected	Pass

Utility Function Tests

Test Case	Input	Expected Output	Actual Result	Status
Email normalization	Mixed-case email	Lowercase email	As expected	Pass
Role conversion	Email input	Correct role output	As expected	Pass
File handling utilities	Sample file input	Correct parsing/output	As expected	Pass
Scheduling calculations	Sample datasets	Correct computed values	As expected	Pass

CSV Export Tests

Test Case	Input	Expected Output	Actual Result	Status
Summary CSV generation	SlotAggregate dataset	Valid summary CSV file	As expected	Pass
Detailed CSV generation	SlotAggregate dataset	Detailed student-level CSV output	As expected	Pass
Recommended CSV generation	Ranked slots	Filtered recommendation CSV	As expected	Pass

C.2 Integration Testing

Integration tests verify correct interaction between system components. While tests are executed in isolation, integration behavior is simulated using mocked dependencies.

Authentication & Role Access

Test Case	Input	Expected Output	Actual Result	Status
Valid login (Student)	Correct credentials	Student session created	As expected	Pass
Valid login (Professor)	Correct credentials	Professor session created	As expected	Pass
Unauthorized access	Null user session	Error thrown	As Expected	Pass
Role-based access control	Student attempting professor route	Access denied	As expected	Pass

Dashboard Data Flow

Test Case	Input	Expected Output	Actual Result	Status
Course data fetch	API request from frontend	Correct course data rendered	As expected	Pass
Scheduling update propagation	Availability submission	Updated rankings reflected in UI	As expected	Pass

Server Action Side Effects

Test Case	Input	Expected Output	Actual Result	Status
Redirect behavior	Action requiring redirect	Redirect triggered	As expected	Pass
Cache revalidation	Create/update/delete actions	revalidatePath called	As expected	Pass
Database interaction	Prisma mock calls	Correct queries executed	As expected	Pass

C.3 Edge Case Testing

Edge cases were tested to ensure system stability under abnormal conditions.

Test Case	Input	Expected Output	Actual Result	Status
Empty database response	No course/user found	Graceful error handling	As expected	Pass
Invalid email format	Malformed email	Error or fallback handling	As expected	Pass
Unauthorized mutation	No authenticated user	Operation rejected	As expected	Pass
Prisma failure simulation	Rejected promise	Custom error thrown	As expected	Pass
Test user detection	Pattern-matching email	Alternate logic path executed	As expected	Pass
Empty course enrollment	No student availability	System returns empty/neutral schedule	As expected	Pass
Tie-breaking in ranking	Equal slot scores	Deterministic ordering applied	As expected	Pass

Invalid email domain login	Unsupported email domain	Authentication rejected	As expected	Pass
----------------------------	--------------------------	-------------------------	-------------	------

Requirement Traceability

Requirement ID	Test Coverage	Status
FR1 (Authentication)	Use server action tests	Pass
FR2 (RBAC)	Auth + protected action tests	Pass
FR5 (Aggregation)	Availability + calculation tests	Pass
FR6 (Ranking)	Scheduling engine tests	Pass
FR7 (CSV Export)	File handling test suite	Pass

C.4 Summary of Results

All major functional and non-functional requirements were successfully validated through structured testing. No critical failures were identified during unit, integration, or edge case testing.

Test Results Figure

```
offic@Command-Center MINGW64 /c/Projects/specify (main)
$ npm run test
> specify@0.1.0 test
> vitest

DEV v4.0.18 C:/Projects/specify

✓ example.test.ts (1 test) 2ms
stdout | lib/__tests__/user-actions.test.ts > ensureUser - test user flow > upserts test user and returns email
Test User detected in Server Action.

✓ lib/__tests__/calculations.test.ts (7 tests) 7ms
✓ lib/__tests__/user-actions.test.ts (4 tests) 6ms
✓ lib/__tests__/fileHandling.test.ts (7 tests) 6ms
stdout | lib/__tests__/course-actions.test.ts > removeInterest > deletes interest and redirects
Interest Removed by Cascade

✓ lib/__tests__/course-actions.test.ts (10 tests) 11ms
✓ lib/__tests__/conversions.test.ts (21 tests) 6ms

Test Files 6 passed (6)
Tests 50 passed (50)
Start at 14:34:42
Duration 1.35s (transform 1.58s, setup 0ms, import 2.67s, tests 37ms, environment 1ms)

PASS Waiting for file changes...
press h to show help, press q to quit
```

All major functional and non-functional requirements were successfully validated through structured testing. No critical failures were identified during unit, integration, or edge case testing.

The system demonstrates:

- Proper authentication and role-based access enforcement
- Reliable user and course management workflows
- Correct execution of server-side business logic through tested server actions
- Correct scheduling aggregation and ranking behavior
- Accurate CSV export generation across all formats
- Stable handling of edge cases, invalid inputs, and simulated failure conditions

All tests passed, confirming that the SpecifY system meets its intended design and functional requirements and that its backend logic is robust, predictable, and production-ready.

Appendix D: Manuals

This appendix provides user and maintenance documentation for the SpecifY Scheduling System. It is intended to support both end users interacting with the system and developers responsible for maintaining or extending the application.

D.1 User Manual

1. Login

- Navigate to the login page
- Authenticate using institutional credentials

2. Browse Courses

- After login, access the course dashboard
- View available courses and their details
- Select courses of interest for participation

3. Submit Availability

- For selected courses, enter preferred availability time slots
- Submit availability data to be processed by the scheduling system

4. View Scheduling Results

- **Students:** View available courses and assigned scheduling outcomes
- **Professors:** View ranked scheduling recommendations generated from aggregated student availability

D.2 Source Code Repository

The source code for the SpecifY Scheduling System is maintained in a GitHub repository:

<https://github.com/ZDowntime/SpecifY>

Access is granted to the instructor via an invitation to the organization that will be sent via email, providing the instructor access to the repository for review and evaluation purposes.